universidade de aveiro  theoria poiesis praxis

Report Lab Work 1

Information and Coding (2026/27)

Work done by:
José Fernandes nº 114472
Rúben Franco nº 114827
Diogo Carvalho nº 113221

Contents

1 Decisions **1**
1.1 Programming Language 1

2 Part 1 **2**
2.1 Exercise 1 2
2.2 Exercise 2 2
2.3 Exercise 3 2
2.4 Exercise 4 2

3 Part 2 **4**
3.1 Methods and Algorithms 4
3.2 Codec Design 4
3.3 Comparison with Other Codecs: 5
3.4 How to run 6
3.4.1 Building 6
3.4.2 Encoder 6
3.4.3 Decoder 6
3.4.4 Running without building 6

4 Part 3 **7**
4.1 Methods and Algorithms 7
4.1.1 The DCT algorithm 7
4.1.2 Block size 7
4.1.3 Coefficient quantization 8
4.1.4 Encoding of runs of zero 8
4.1.5 Noise 8
4.2 Final result and comparison with other codecs 9

Abbreviations

DCT discrete cosine transform

IEEE Institute of Electrical and Electronics Engineers

JPEG Joint Photographic Experts Group

MSE mean squared error

SNR signal-to-noise ration

VBR variable bit rate

Chapter 1

Decisions

1.1 Programming Language

The programming language used to for this work was Go. The main reasons why this was the language chosen are the shared ease of development by each member of the group, and the accessibility of lightweight libraries. Because Go is a midterm between C and C# we were able to maintain a good level of abstraction while keeping things simple and efficient.

Chapter 2

Part 1

2.1 Exercise 1

The histogram exercise generate CSV files for each channel with two columns: a start value of the bin, and the count of the values.

2.2 Exercise 2

The quantizer program simply subtracts bits from all the samples in a WAVE file, a writes the result to the quantized.wav file.

2.3 Exercise 3

The `wav_cmp` program compares a modified WAV file with the original. It prints the results per channel and for the mono average. The metrics are:

- **Mean squared error (L2 norm):** the average of the squared difference between original and modified samples.
- **Maximum absolute error (L-infinity norm):** the largest absolute difference between any original and modified sample.
- **Signal-to-noise ratio:** the energy of the original signal divided by the energy of the error, in dB.

The mono values are computed on the channel average (L+R)/2, so the mono error is the error of the averaged signal. The program checks that both files have the same bit depth, sample rate, channel count and number of samples before comparing them, and it reads the files in blocks of 8192 frames to keep memory use low.

2.4 Exercise 4

As required for exercise 4 the following functions were implemented.

- **Echo:**

  The Echo function is done by iterating the audio buffer and adding a percentage of the last sample of the same channel to the current sample. The coefficient must be less than 1 (less than 100% of the last sample) so that the audio fades instead of growing louder and noisier. One important aspect of this function is that the echo added to the current sample is added by an original old sample with no echo itself.

- **Multiple Echos:**

  Much like the Echo function, this effect is done by iterating the audio buffer and adding a percentage of the last sample of the same channel to the current sample. The only difference is that if the sample being analysed is not the first one, the added sample for the echo will by an "echoed" sample itself. The coefficient rule is the same as in the Echo function.

- **Amplitude Modulation:**

  Each sample is multiplied by the corresponding sample of a constant frequency sine wave. This is not to be confused the amplitude modulation used to transmit a audio signal through radio.

- **Time-varying Delay:**

  Regarding Time-varying Delay, the function is done by simply adding a user chosen delay to each sample. The buffer data array size must also be enlarged by the delay value to accommodate for the sample time jump.

Chapter 3

Part 2

3.1 Methods and Algorithms

- **Differential Coding:** The Differential Coding is a technique that replaces each symbol for the difference between the current sample and a predicted value, typically the immediately preceding sample. In the current project the each symbol is a sample from the audio buffer, and the delta is calculated with the difference between the current sample and the next sample in the same audio channel. Because adjacent data points in continuous signals are often highly correlated, their differences are significantly smaller in magnitude than the raw absolute values, requiring fewer bits to encode.
- **Huffman Coding:** Huffman Coding is a lossless encoding algorithm that compresses data by assigning variable-length binary codes to symbols based on the frequency of appearances. In this case, the symbols can be the sample values from the buffer or the values created by the differential coding. Symbols that appear frequently receive shorter bit sequences, while rarer symbols receive longer ones, minimizing the overall bitstream length.
- **Blocking/Slicing:** Slicing divides the data into blocks of the same size, usually except the last one, which holds the remaining samples. Each block has its own Huffman table, so the codes can adapt to changes in the signal along the file. The cost is that every block must store its own table, which is significant for small blocks. The block size (4096 samples) is stored in the header of the compressed file.
- **LZSS Coding (unused):** LZSS is a dictionary-based method that replaces sequences that already appeared earlier in the data with a reference (offset and length) to the earlier occurrence. We implemented it, but it was not included in the final codec because the gain did not justify the extra complexity. The code is kept in the `lzss` folder of the repository.

3.2 Codec Design

The codec is made of two programs written in Go, `wav_lossless_enc` and `wav_lossless_dec`.

File format: The compressed file starts with a 19-byte header: number of channels, bit depth, sampling rate, number of frames, a byte of flags (differential coding, slicing) and the block size. The number of channels and the bit depth are stored minus one. The header is followed by one block, or one block per slice, each made of a Huffman table and the coded samples. Samples are written most significant bit first and each block is padded to a whole byte.

Encoder: The encoder reads the WAV file, optionally applies differential coding, optionally splits the samples into blocks, and for each block counts the frequency of each value, builds the Huffman tree with a priority queue, derives the canonical codes and writes the table and the coded bits.

Decoder: The decoder reads the header and, for each block, rebuilds the codes from the table. It reads the bits one at a time and checks, for each code length, whether the bits read so far fall inside the range of codes of that length, so no tree is needed. Finally, if differential coding was used, it adds the previous sample of the same channel back, from the first sample to the last, and writes the WAV file.

Limitations: Only stereo files are supported. The decoder writes a standard WAV header, so extra metadata chunks of the original file are not preserved, although the samples are identical.

3.3 Comparison with Other Codecs:

The codec was compared with FLAC (specialised for audio) and with the general-purpose compressors zstd, xz (LZMA) and bzip2, using their highest common compression levels. All tools were run on the same seven audio samples, and every tool reproduced the original audio exactly. Table 3.1 shows the totals over all samples and Table 3.2 shows the compression ratio of each sample.

		Time (s)		Peak memory (MB)	
Tool	Ratio	Comp.	Decomp.	Comp.	Decomp.
Ours	1.162	4.38	0.56	92	36
Ours + differential	1.441	4.34	0.52	92	40
Ours + slicing	0.680	7.00	0.54	102	49
Ours + diff. + slicing	0.973	5.91	0.51	103	42
FLAC `-8`	1.786	0.36	0.11	3	2
zstd `-19`	1.119	1.05	0.04	96	10
xz `-9e`	1.200	2.25	0.60	120	16
bzip2 `-9`	1.172	1.18	0.64	7	5

Table 3.1: Totals over all samples. Ratio is the total original size divided by the total compressed size, times are the sum over all samples and memory is the largest peak of any sample.

Tool	S1	S2	S3	S4	S5	S6	S7	Avg.
Ours	1.129	1.195	1.180	1.080	1.248	1.336	1.004	1.167
Ours + differential	1.355	1.384	1.470	1.361	1.618	1.851	1.177	1.460
Ours + slicing	0.629	0.719	0.693	0.629	0.796	0.845	0.544	0.694
Ours + diff. + slicing	0.830	0.964	1.042	0.861	1.271	1.629	0.687	1.041
FLAC `-8`	1.513	1.603	1.798	1.761	2.154	3.031	1.376	1.891
zstd `-19`	1.065	1.127	1.118	1.077	1.247	1.214	1.020	1.124
xz `-9e`	1.110	1.204	1.211	1.112	1.360	1.450	1.036	1.212
bzip2 `-9`	1.149	1.162	1.203	1.087	1.286	1.309	1.021	1.174

Table 3.2: Compression ratio of each sample (S1 to S7 are `sample01` to `sample07`) and the average of the seven ratios.

Compression ratio: Differential coding is the most important choice: it raises the average ratio of our codec from 1.167 to 1.460, and it helps on every sample. With it, our codec is better than zstd, xz and bzip2 on all seven samples, but FLAC is better still (1.891 on average). This is expected, since FLAC predicts each sample with linear prediction and codes the residual, while

we only use the previous sample as the prediction. Slicing makes the results worse (average ratios of 0.694 and 1.041): every block pays for its own table, and with blocks of 4096 samples this is more than what adapting to the signal saves, so without differential coding the compressed files are larger than the originals.

Speed and memory: Our codec is much slower than FLAC when compressing (about 4.3 s against 0.36 s) and about five times slower when decompressing, and it needs far more memory (92 MB against 3 MB). The reason is that it loads the whole file as a list of integers and processes it at once, while FLAC works on small frames. It is, however, comparable to xz in time and memory, and the speed is dominated by the bit-by-bit Huffman coding and decoding. Decompression is cheaper than compression because the decoder does not have to count symbols or build the tree. zstd is the fastest at decompressing (0.04 s).

3.4 How to run

The programs are built from the root of the repository, where the `go.mod` file is. The encoder is in the `enc` folder and the decoder in the `dec` folder.

3.4.1 Building

```
go build -o wav_lossless_enc ./enc
go build -o wav_lossless_dec ./dec
```

3.4.2 Encoder

The `-differential` and `-slicing` flags are optional and can be used together or individually. Without flags, the Huffman algorithm is applied directly to the audio buffer.

```
./wav_lossless_enc [-differential] [-slicing] input.wav compressed_file
```

3.4.3 Decoder

The decoder reads the flags from the header of the compressed file, so it needs none.

```
./wav_lossless_dec compressed_file output.wav
```

3.4.4 Running without building

```
go run ./enc [-differential] [-slicing] input.wav compressed_file
go run ./dec compressed_file output.wav
```

Chapter 4

Part 3

4.1 Methods and Algorithms

4.1.1 The DCT algorithm

As per the exercise guide, we used the discrete cosine transform (DCT) as the base of our compression algorithm. The DCT takes a vector of real valued samples, and returns a vector of real amplitudes of cosine waves of increasing frequencies, which we will call coefficients. Both representations are mathematically equivalent: given the coefficients, it is possible to get the vector of samples without error.

In practice, compression algorithms that use the DCT act on integer valued samples, but the coefficients are, most likely, not integers. This is a problem for computers, as bounded integers are represented perfectly, but real numbers are not. There are two ways to solve this problem:

- Implement the DCT in terms of integer values: Some JPEG encoders implement the DCT only with integer arithmetic. This also has the benefit of being deterministic: the encoder will produce the same result on different computing systems;
- Use a type of numbers to approximate the reals: Consumer hardware supports what are called floating point numbers. Every calculation with the will add error. These numbers can be used if, at the end, the accumulated error is small enough.

Another problem to consider is the running time of the DCT algorithm we choose. The naive algorithm runs in $O(n^2)$, and would run too slow. Unfortunately, we could not implement a faster algorithm ourselves.

Given these two constraints, we chose to use a library[1] that implements a fast DCT algorithm that runs in $O(nlog(n))$ time by Byeong Gi Lee. This library acts on Go's float64 type, which is a IEEE-754 double precision floating point number type, also known as binary64.

4.1.2 Block size

The DCT has another parameter to adjust: the vector size. The block size is also the amount of coefficients that work together to give a vector of samples back. The DCT presents a property called energy compaction. This says that for certain input vectors, the high frequency coefficients will be closer to 0 than lower frequency coefficients. This was empirically tested by running the encoder with higher and higher block sizes. Even at small block sizes, the property was visible from the resulting values, as seen in figure 4.1.

[1] `https://github.com/zikichombo/dsp/tree/4acf7f64302920eb2e01a427a17cf8ee8527f7c6/dct`

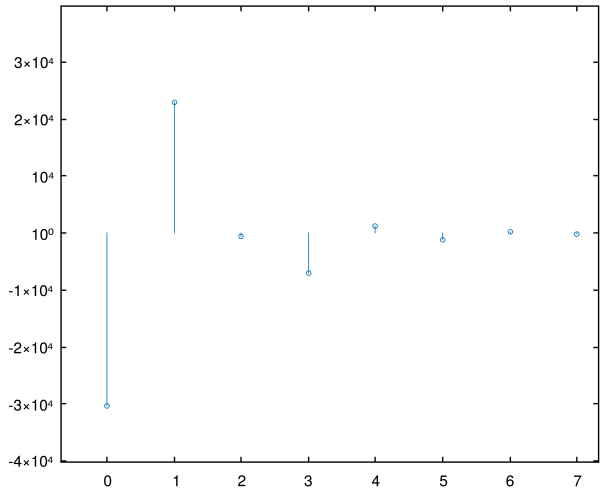


Figure 4.1: The coefficients for the DCT applied on the first 8 samples of the sample01.wav file

4.1.3 Coefficient quantization

The next problem we solved was how to store the coefficients. We first attempted to convert the coefficients to 16 bit floating point numbers. Two problems arose: for the block size we chose for testing, the values grew into infinities. We solved this by scaling the coefficients in the encoded and the decoder. However, this storage mechanism does not compress the data, as the size of the coefficients is the same as the samples.

Our second attempt was much more aggressive: round all the coefficients to integer values, store the maximum absolute value in a block, and map them linearly to a signed 8 bit number relative to how close each coefficient is to the maximum. This cuts the file size by roughly half.

4.1.4 Encoding of runs of zero

As a result of the previous step, a lot of coefficients get truncated into the value zero. An efficient way to represent these coefficients is to store non zero coefficients as themselves, and then the zeros as a single number. In practice we observed that storing single zeros inline with other coefficients actually resulted in smaller file sizes, so zeros are only encoded as a single number if two or more appear in a row.

4.1.5 Noise

The aggressive coefficient quantization step unfortunately adds audible noise. This is most noticeable in blocks where there are both loud and quiet parts in the audio. We theorized this happened because of the relatively huge chunk size of 8192 we used to test the compression. With a fully implemented codec, we could now test various block sizes.

As a target, we compressed a sample audio file with the reference Opus encoder from opus-tools 0.2[2], with a target variable bit rate (VBR) of 128kbps[3], which the Xiph.org wiki says is "pretty much transparent". Such a file had a signal-to-noise ration (SNR) of around 19 dB, a mean squared error (MSE) of around 200 000, and a maximum error of around 3250. We then found was that our codec, although producing worse noise, is technically better than Opus in all those measurements.

[2] `https://opus-codec.org/release/dev/2018/09/18/opus-tools-0_2.html`

[3] opusenc –vbr –music –comp 10

4.2 Final result and comparison with other codecs

The table below help to analyse the processing time, sizes, and compression ratios between different lossless encoding codecs.

Metric	**Ours**	**Opus**	**MP3**	**QOA**
Encoding Time (seconds)	0.56	0.97	2.02	0.86
Compression Ratio (%)	28.0	9.0	9.0	20.2

Table 4.1: Performance and compression metrics across different encoding configurations